

Does the model answer from the memory I gave it, or from what it already knew?

Amrit Khadka
M.Sc. Computer Science, TU Darmstadt

What I built

I wrote a small synthetic memory of 19 short facts about a fictional research institute. Six are ordinary invented facts the model cannot possibly know, like which floor a unit sits on. Three deliberately contradict something the model does know, such as “in this setting, the capital of Australia is Sydney”. Two more are what I called bridge facts: they refer to an overridden fact without naming it, so answering them correctly requires the override to carry through a step of reasoning. The last four are pairs of entries that contradict *each other*, one stale and one current, which is the situation a real agent memory ends up in after a few months.

On top of that I wrote 20 queries across eight difficulty tiers, and I wrote down the expected answer for each one before running anything.

Two things varied. The first was how much memory the model got: nothing at all, only the entries needed for that question, the whole memory, the whole memory plus six unrelated sentences, or the whole memory plus six sentences chosen to sound confusingly similar to the real ones. The last two are matched on length, so if they differ, the cause is confusion rather than sheer volume. The second was a single sentence in the system prompt describing how much authority the memory has.

680 calls on Groq’s free tier, about an hour, no errors and nothing truncated. The main model was `llama-3.1-8b-instant`; `llama-3.3-70b` and `qwen3.6-27b` ran on a subset. All confidence intervals bootstrap over queries rather than rows, because three samples of the same question are not three independent observations.

The first version didn’t work, and fixing it taught me the most

My first attempt put the memory’s authority into the system prompt as a fixed instruction: the memory “takes precedence over anything you believe otherwise”. I ran a 12-call pilot on the hardest override question. Eleven of twelve answered Sydney, even with three confusingly similar names competing in the context.

That instruction was doing the work. I had told the model how to resolve the conflict, then measured whether it resolved the conflict. So I turned that sentence into a variable instead of a constant, with a neutral version that simply says the memory “contains notes about this setting”.

A second 48-call pilot showed something else. Every tier was at ceiling except the stale-versus-current pairs, which came out close to a coin flip. That was the only place the models were actually struggling, so I expanded it from one question into eight, crossing four different ways of signalling which entry is current against the order the two entries appear in. I also had to pin that order, because until then my shuffle was randomising it, which meant the one variable that matters most for stale memory was being set by chance.

Question 1: memory or prior?

Here is the result I did not expect. Across all 680 responses, the model answered from its prior exactly zero times.

That sounds like a clean win for the memory, and I nearly wrote it up that way. But the zero also holds in the no-memory condition, where the model had nothing to work from and should have said Canberra, or 100 degrees, or Everest. It didn't. That condition scored 17% correct, which is exactly two out of twelve: the two questions whose right answer was "I don't know". So with no memory the model abstained on everything instead of falling back on what it knows.

The reason is a rule I put in my own prompt: if the memory doesn't answer the question, reply **NOT IN MEMORY**. That instruction suppressed prior-answering entirely. So I can't claim the memory beat the prior. All I can honestly say is that the prior never got a chance to compete, because I accidentally told the model not to use it.

This also means my fictional-versus-contradicting comparison isn't informative. Override questions scored 66% and purely fictional ones 58%, which is the opposite of what a prior-intrusion story predicts. Every failure I actually observed was a *retrieval* failure: the model either gave up, lost track partway through a multi-step chain, or picked the wrong entry when two conflicted. For a model of this generation, accepting a made-up fact turns out to be the easy part. Finding the right fact in a cluttered memory is the hard part.

Question 2: does delivery format matter, or only size?

what the model was given	answered from memory	95% CI	prompt tokens
nothing	17%	[0, 42]	118
only the relevant facts	92%	[81, 100]	146
the whole memory	81%	[61, 96]	436
whole memory + unrelated padding	74%	[56, 89]	507
whole memory + similar padding	61%	[39, 85]	526

The interesting comparison is the bottom two rows. They contain the same real information and differ by 19 tokens, about 4%, but they differ by 12 percentage points in accuracy. If extra length were the problem, those two should look the same. They don't, which suggests the damage comes from entries that *sound like* the answer, not from context volume. "Marek Voss" and "Tomas Aldrich" cost the model far more than six sentences about the cafeteria and the bike racks.

I want to be careful here though. Those two intervals overlap a lot. It is a suggestive point estimate, not a demonstrated effect, and with twelve questions at three samples each I could only have detected something enormous.

The authority sentence made no difference I can defend. Neutral came out at 65% and authoritative at 55%, with intervals that almost completely overlap. What did shift was the shape of the failures rather than the amount: the authoritative framing produced more abstentions (47% against 39%) rather than more wrong answers. If that's real, it means telling a model its memory is authoritative makes it *stricter* about answering at all, which is not what I assumed and is worth testing on its own. To detect a ten-point difference here I'd need roughly four or five times the items, or a continuous measure instead of a three-sample proportion.

Sorted by tier, ignoring the no-memory baseline, the picture is:

tier	correct	95% CI	questions
three-hop chain (fictional only)	21%	[21, 21]	1
stale vs. current entries	35%	[21, 50]	8
ambiguous question	50%	[50, 50]	1
two-hop through an override	76%	[58, 92]	3
direct lookup	92%	[88, 96]	3
unanswerable / underspecified	96%	[96, 96]	2

The tiers with one question have meaningless intervals and should be read as anecdotes. The 96% on unanswerable questions looks impressive until you remember I told the model the exact phrase to use, so it measures obedience more than judgement. The 4% that failed is the part worth reading: one answer confidently invented “Dr. Priya Raman leads the Artificial Intelligence Development team” for a person who appears nowhere in the memory or the distractors. That single response is a useful reminder of how much to trust the other numbers.

The staleness result, and a mistake in my own design

This was meant to be the centrepiece: four ways of marking which of two conflicting entries is current, to see which one a model can actually use. What I found instead was a flaw in how I wrote the pairs.

stale entry	current entry	signal	correct
“ is stationed at Northfield”	“ was relocated”	one has a date	15%
“ is in the basement”	“ was moved”	both have dates	25%
“ are issued by Records Annex”	“ are now issued”	“this replaces”	33%
“ is Ivo Brandt”	“ is now Sanne”	the word “now”	67%

Look at the verbs. I wrote every stale entry in the present tense, and in the top two rows I wrote the current entry in the past tense. “Kestrel-7 *is* stationed at Northfield” reads like a standing fact; “*was* relocated” reads like something that happened once. Grouped that way, pairs whose current entry is past tense score 20% [9, 29] and pairs whose current entry is present tense score 50% [32, 70].

So the tempting conclusion, that timestamps don’t help models resolve stale memory, is not one I can support. Tense and date-presence line up perfectly in my four pairs, so there is no way to tell which is responsible. I’d rather say that plainly than report a clean finding I can’t back up.

Two things I can say. Order made no difference at all: current entry first scored 34%, current entry last 35%, which contradicts what a lost-in-the-middle effect would predict. And on the cells all three models saw, size mattered enormously: the 8B model managed 33% [4, 62] while the 70B and the 27B both scored 100%.

What I don’t trust, and what I’d do next

The biggest problem is the one in Question 1. My abstention instruction stopped the model answering from prior even when it had nothing else, so the central comparison never really happened. Everything downstream of that is about retrieval, not about memory versus training.

After that: Groq exposes no log-probabilities, so my measure is the fraction of three samples that matched, which tracks answer stability rather than the model’s actual confidence and is very coarse. The staleness pairs confound tense with dates. Only the 8B model ran the full design, so the model comparisons rest on 32 to 48 observations each. I used one shuffle seed, so position effects are unmeasured. One of my supersession entries says “this replaces the one above”, which is

nonsense in the condition where it appears first. And scoring is rule-based, with 33 rows (4.9%) that I read and judged myself; the answers to “where does Voss work?” that said “the Meridian Institute” I marked underspecified rather than right or wrong, because they’re true but not at the granularity the question asked for.

The follow-up is one experiment with two fixes. First, remove the abstention instruction from the baseline and probe the prior directly, so that prior-answering is demonstrated before I claim anything beat it. That part is fifteen calls. Second, cross date-presence against tense properly as a two-by-two (dated or not, current entry past or present tense), holding the entity and question fixed, with four item sets per cell. That’s the smallest design that separates the two explanations I’m stuck between.

I’d run it on something that exposes log-probabilities, either a local Qwen-7B or an OpenAI-compatible endpoint, so the measure becomes $P(current)/[P(current) + P(stale)]$. A continuous score gives far more information per item than counting three samples, which is directly what my null result on framing was missing. I’d track adherence, abstention and confabulation separately, since this run showed they move independently. Roughly 150 prompts and 500 forward passes: under an hour on one GPU, or about 200K tokens and twenty cents on a paid API.

If there were time for a second one, I’d go back to the interference result. It rests on a single set of six distractor sentences at a single ordering, with overlapping intervals. Running it across five seeds and three distractor pools of increasing similarity would show whether that 12-point gap scales with how similar the distractors are, or whether I just happened to write six unusually confusing sentences.

Files

<code>agentic_memory_experiment.ipynb</code>	the full experiment, runs top to bottom
<code>answer_key.json</code>	the memory, the queries, and expected answers
<code>prompts.jsonl</code>	all 200 prompts as sent
<code>raw_outputs.jsonl</code>	all 680 responses, unedited
<code>scored.csv</code>	per-response codes and review flags
<code>summary.csv</code>	rates by model, condition, framing and tier

The notebook keeps an appendix with the two pilot runs described above, since the design changes only make sense alongside the results that prompted them.